

Advanced MySQL Self-Study Manual — Part 2: Practice Database, Exercises, Answers, Labs

PART 22 — PRACTICE DATABASE: advanced_sql_lab

Run this script as-is in MySQL 8+. It intentionally includes NULLs, duplicates, ties, orphaned rows, and multi-row histories so every pattern in Part 1 has something real to bite into.

```
DROP DATABASE IF EXISTS advanced_sql_lab;
CREATE DATABASE advanced_sql_lab;
USE advanced_sql_lab;

-- ===== DEPARTMENTS =====
CREATE TABLE departments (
    department_id INT PRIMARY KEY AUTO_INCREMENT,
    department_name VARCHAR(100) NOT NULL,
    active TINYINT DEFAULT 1
);

INSERT INTO departments (department_name, active) VALUES
('Engineering', 1), ('Sales', 1), ('Marketing', 1),
('HR', 1), ('Finance', 1), ('Support', 1), ('Legacy Ops', 0);

-- ===== EMPLOYEES =====
CREATE TABLE employees (
    employee_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    department_id INT NULL,           -- NULL: unassigned employee
    manager_id INT NULL,             -- NULL: top of hierarchy
    salary DECIMAL(10,2) NOT NULL,
    hire_date DATE NOT NULL,
    gender CHAR(1),
    FOREIGN KEY (department_id) REFERENCES
departments(department_id),
    FOREIGN KEY (manager_id) REFERENCES employees(employee_id)
);

INSERT INTO employees (name, department_id, manager_id, salary,
hire_date, gender) VALUES
('Alice Chen', 1, NULL, 145000, '2018-01-15', 'F'),
('Bob Martinez', 1, 1, 98000, '2019-03-01', 'M'),
('Carol White', 1, 1, 98000, '2019-06-10', 'F'),    -- tie with Bob
('David Kim', 1, 2, 76000, '2021-02-20', 'M'),
('Eve Torres', 1, 2, 82000, '2020-11-05', 'F'),
('Frank Liu', 2, NULL, 130000, '2017-08-01', 'M'),
('Grace Park', 2, 6, 71000, '2022-01-10', 'F'),
('Henry Adams', 2, 6, 71000, '2022-01-10', 'M'),    -- exact tie:
hire_date + salary
('Isla Rivera', 3, NULL, 110000, '2016-05-12', 'F'),
('Jack Nguyen', 3, 9, 62000, '2023-04-01', 'M'),
('Kara Diaz', 4, NULL, 95000, '2015-09-01', 'F'),
('Liam Foster', 4, 11, 58000, '2023-07-15', 'M'),
('Mia Cooper', 5, NULL, 120000, '2014-03-01', 'F'),
('Noah Bennett', 5, 13, 67000, '2021-10-01', 'M'),
```

```

        ('Olivia Stone', 6, NULL, 60000, '2022-09-01', 'F'),
        ('Pedro Alvarez', NULL, NULL, 55000, '2024-01-01', 'M'), --
unassigned dept
        ('Quinn Osei', 1, 1, 76000, '2021-02-20', NULL); -- unknown
gender

-- ===== EMPLOYEE HISTORY (salary changes) =====
CREATE TABLE employee_history (
    history_id INT PRIMARY KEY AUTO_INCREMENT,
    employee_id INT NOT NULL,
    old_salary DECIMAL(10,2),
    new_salary DECIMAL(10,2),
    changed_at DATE NOT NULL,
    FOREIGN KEY (employee_id) REFERENCES employees(employee_id)
);

INSERT INTO employee_history (employee_id, old_salary, new_salary,
changed_at) VALUES
(2, 85000, 98000, '2021-01-01'),
(2, 98000, 98000, '2022-01-01'), -- no-op change on purpose
(4, 70000, 76000, '2022-06-01'),
(7, 65000, 71000, '2023-01-01'),
(10, 55000, 62000, '2024-01-01');

-- ===== CUSTOMERS =====
CREATE TABLE customers (
    customer_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(150),
    signup_date DATE NOT NULL,
    status VARCHAR(20) DEFAULT 'ACTIVE'
);

INSERT INTO customers (name, email, signup_date, status) VALUES
('Acme Corp', 'billing@acme.test', '2020-01-10', 'ACTIVE'),
('Beta LLC', 'ap@beta.test', '2020-03-15', 'ACTIVE'),
('Gamma Inc', 'finance@gamma.test', '2021-06-01', 'INACTIVE'),
('Delta Co', 'orders@delta.test', '2021-09-20', 'ACTIVE'),
('Epsilon Ltd', NULL, '2022-02-01', 'ACTIVE'), -- missing
email
('Zeta Group', 'hello@zeta.test', '2023-01-05', 'ACTIVE'),
('Eta Partners', 'info@eta.test', '2023-08-11', 'ACTIVE'),
('Theta Ventures', 'contact@theta.test', '2024-04-01', 'ACTIVE'),
('NoOrders Inc', 'x@noorders.test', '2024-05-01', 'ACTIVE'); --
deliberately has zero orders

-- ===== CATEGORIES =====
CREATE TABLE categories (
    category_id INT PRIMARY KEY AUTO_INCREMENT,
    category_name VARCHAR(100) NOT NULL
);

INSERT INTO categories (category_name) VALUES
('Electronics'), ('Office Supplies'), ('Furniture'), ('Software');

-- ===== PRODUCTS =====
CREATE TABLE products (
    product_id INT PRIMARY KEY AUTO_INCREMENT,
    product_name VARCHAR(100) NOT NULL,
    category_id INT,
    price DECIMAL(10,2) NOT NULL,
    FOREIGN KEY (category_id) REFERENCES categories(category_id)
);

INSERT INTO products (product_name, category_id, price) VALUES
('Laptop Pro', 1, 1200.00), ('Wireless Mouse', 1, 25.00), ('4K

```

```

Monitor', 1, 350.00),
('Desk Lamp', 2, 18.00), ('Notebook Pack', 2, 8.50), ('Stapler', 2,
6.00),
('Office Chair', 3, 220.00), ('Standing Desk', 3, 450.00),
('CRM License', 4, 99.00), ('Analytics Suite', 4, 199.00);

-- ===== ORDERS =====
CREATE TABLE orders (
    order_id INT PRIMARY KEY AUTO_INCREMENT,
    customer_id INT,
    order_date DATE NOT NULL,
    status VARCHAR(20) DEFAULT 'COMPLETED',
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
);

INSERT INTO orders (customer_id, order_date, status) VALUES
(1, '2024-01-05', 'COMPLETED'), (1, '2024-02-10', 'COMPLETED'), (1,
'2024-03-01', 'CANCELLED'),
(2, '2024-01-15', 'COMPLETED'), (2, '2024-01-20', 'COMPLETED'), (2,
'2024-04-01', 'COMPLETED'),
(3, '2023-12-01', 'COMPLETED'),
(4, '2024-02-01', 'COMPLETED'), (4, '2024-02-02', 'COMPLETED'), --
consecutive-day orders
(4, '2024-02-03', 'COMPLETED'),
(5, '2024-03-10', 'COMPLETED'),
(6, '2024-05-01', 'COMPLETED'), (6, '2024-05-15', 'PENDING'),
(7, '2024-06-01', 'COMPLETED'),
(NULL, '2024-07-01', 'COMPLETED'); -- orphaned order, no matching
customer

-- ===== ORDER ITEMS =====
CREATE TABLE order_items (
    order_item_id INT PRIMARY KEY AUTO_INCREMENT,
    order_id INT NOT NULL,
    product_id INT NOT NULL,
    quantity INT NOT NULL,
    unit_price DECIMAL(10,2) NOT NULL,
    FOREIGN KEY (order_id) REFERENCES orders(order_id),
    FOREIGN KEY (product_id) REFERENCES products(product_id)
);

INSERT INTO order_items (order_id, product_id, quantity, unit_price)
VALUES
(1, 1, 1, 1200.00), (1, 2, 2, 25.00),
(2, 3, 1, 350.00),
(4, 1, 1, 1200.00), (4, 9, 3, 99.00),
(5, 4, 5, 18.00),
(6, 7, 1, 220.00), (6, 8, 1, 450.00),
(7, 9, 1, 99.00),
(8, 5, 10, 8.50), (9, 6, 4, 6.00), (10, 5, 2, 8.50),
(11, 10, 1, 199.00),
(12, 2, 1, 25.00), (13, 2, 1, 25.00),
(14, 9, 2, 99.00);

-- ===== PAYMENTS =====
CREATE TABLE payments (
    payment_id INT PRIMARY KEY AUTO_INCREMENT,
    order_id INT NOT NULL,
    amount DECIMAL(10,2) NOT NULL,
    paid_at DATE,
    FOREIGN KEY (order_id) REFERENCES orders(order_id)
);

INSERT INTO payments (order_id, amount, paid_at) VALUES
(1, 1250.00, '2024-01-06'), (2, 350.00, '2024-02-11'),
(4, 1497.00, '2024-02-02'), (5, 90.00, '2024-03-11'),

```

```

(6, 670.00, '2024-05-02'), (7, 99.00, '2024-06-02'),
(8, 85.00, NULL); -- payment recorded but not yet cleared

-- ===== ACCOUNTS / TRANSACTIONS (for transaction-demo
queries) =====
CREATE TABLE accounts (
    account_id INT PRIMARY KEY AUTO_INCREMENT,
    customer_id INT,
    balance DECIMAL(10,2) NOT NULL DEFAULT 0,
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
);

INSERT INTO accounts (customer_id, balance) VALUES (1, 5000.00), (2,
3200.00), (4, 800.00);

CREATE TABLE transactions (
    transaction_id INT PRIMARY KEY AUTO_INCREMENT,
    account_id INT NOT NULL,
    amount DECIMAL(10,2) NOT NULL,
    transaction_type VARCHAR(20),
    created_at DATETIME,
    FOREIGN KEY (account_id) REFERENCES accounts(account_id)
);

INSERT INTO transactions (account_id, amount, transaction_type,
created_at) VALUES
(1, -100.00, 'TRANSFER_OUT', '2024-06-01 10:00:00'),
(2, 100.00, 'TRANSFER_IN', '2024-06-01 10:00:01'),
(1, -50.00, 'WITHDRAWAL', '2024-06-05 14:30:00'),
(4, 200.00, 'DEPOSIT', '2024-06-06 09:15:00');

-- ===== CUSTOMER STATUS HISTORY (for latest-status pattern)
=====
CREATE TABLE customer_status_history (
    id INT PRIMARY KEY AUTO_INCREMENT,
    customer_id INT NOT NULL,
    status VARCHAR(20) NOT NULL,
    changed_at DATETIME NOT NULL,
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
);

INSERT INTO customer_status_history (customer_id, status,
changed_at) VALUES
(3, 'ACTIVE', '2021-06-01 00:00:00'),
(3, 'INACTIVE', '2023-01-01 00:00:00'),
(6, 'ACTIVE', '2023-01-05 00:00:00'),
(6, 'ACTIVE', '2023-06-01 00:00:00'); -- duplicate status re-
recorded, no real change

```

What's deliberately embedded for you to find: -

employees.department_id NULL → Pedro is unassigned (for anti-join / orphan practice). - employees.manager_id NULL → 6 department heads at the top of the hierarchy. - Exact ties: Bob & Carol (same salary); Grace & Henry (same salary *and* hire date). - orders.customer_id = NULL → one truly orphaned order. - customers "NoOrders Inc" → zero matching orders (anti-join target). - Consecutive-day orders for Delta Co (order_id 8, 9, 10) → gaps-and-islands practice. - payments.paid_at IS NULL → an unpaid/pending payment row. - employee_history row with old_salary = new_salary → a no-op change to test your filtering logic. - customer_status_history duplicate ACTIVE entries → tests "find the actual latest distinct status."

PART 23 — PRACTICE EXERCISES

(attempt before checking Part 24 answers)

Level 1 — Fundamentals (10)

1. List all employees with no assigned department.
2. Find all customers whose status is 'INACTIVE'.
3. List products priced above \$100, ordered by price descending.
4. Count how many employees are in each department.
5. Find the total quantity ordered for each product.
6. List all orders with a NULL customer.
7. Find employees hired before 2020.
8. List distinct order statuses present in the orders table.
9. Find the average salary across all employees.
10. List customers who have no email on file.

Level 2 — Joins, Grouping, Basic Subqueries (10)

11. List each employee with their department name (include employees with no department).
12. Find departments with more than 3 employees.
13. Find customers who have never placed an order (two different techniques).
14. Find the total revenue (quantity * unit_price) per order.
15. Find employees who earn more than the company-wide average salary.
16. List each order with its customer's name and the number of items in that order.
17. Find products that have never been ordered.
18. List each department along with its highest-paid employee's salary.
19. Find customers who placed more than one order.
20. Find the employee(s) who report directly to "Alice Chen."

Level 3 — Window Functions, CTEs, Advanced Patterns (10)

21. Rank employees by salary within their department using DENSE_RANK.
22. Find the second-highest salary overall (handle ties correctly).
23. For each customer, find their most recent order date using ROW_NUMBER.
24. Write a recursive CTE to output the full management chain (employee → manager → manager's manager...) for "Jack Nguyen."
25. Find the running total of payment amounts ordered by paid_at.
26. Identify employees who share the exact same salary as at least one colleague (duplicate salary detection).
27. Find the month-over-month change in number of orders placed (group orders by month first).
28. Find Delta Co's longest streak of consecutive-day orders (gaps-and-islands).
29. Using employee_history, find the most recent salary change per employee, excluding no-op changes (old_salary = new_salary).
30. Find each customer's latest distinct status from customer_status_history (watch for the duplicate re-recorded status).

Level 4 — Interview-style (10)

31. Find departments where the average salary exceeds the company-wide average salary.
32. Find the top 2 highest-priced products in each category.

33. Find customers who bought every product in the “Office Supplies” category (relational division).
 34. Write a query to detect the exact-duplicate employee pair (same salary AND same hire date).
 35. Find all orders where the sum of order_items doesn’t match the corresponding payments.amount (reconciliation).
 36. Rewrite this query to be sargable: `SELECT * FROM orders WHERE YEAR(order_date) = 2024;`
 37. Explain (in words) what EXPLAIN would likely show for `SELECT * FROM employees WHERE department_id = 3;` with and without an index on department_id.
 38. Find employees who have never had a salary change recorded in employee_history.
 39. Write a query using NOT EXISTS to find products never included in any order — then explain why NOT IN would be risky here if order_items.product_id could contain NULLs.
 40. Find the percentage each department’s total salary contributes to the company’s total salary spend.
-

PART 24 — ANSWER KEY

(Solutions below. Attempt first — checking early defeats the point.)

1.

```
SELECT * FROM employees WHERE department_id IS NULL;
```

Expected: Pedro Alvarez.

2.

```
SELECT * FROM customers WHERE status = 'INACTIVE';
```

Expected: Gamma Inc.

3.

```
SELECT * FROM products WHERE price > 100 ORDER BY price DESC;
```

4.

```
SELECT department_id, COUNT(*) FROM employees GROUP BY department_id;
```

Note: this includes a NULL group for Pedro — expected behavior, worth noticing.

5.

```
SELECT product_id, SUM(quantity) AS total_qty FROM order_items GROUP BY product_id;
```

6.

```
SELECT * FROM orders WHERE customer_id IS NULL;
```

7.

```
SELECT * FROM employees WHERE hire_date < '2020-01-01';
```

8.

```
SELECT DISTINCT status FROM orders;
```

9.

```
SELECT AVG(salary) FROM employees;
```

10.

```
SELECT * FROM customers WHERE email IS NULL;
```

11.

```
SELECT e.name, d.department_name
FROM employees e LEFT JOIN departments d ON d.department_id =
e.department_id;
```

LEFT JOIN is required — an INNER JOIN would silently drop Pedro.

12.

```
SELECT department_id, COUNT(*) AS cnt FROM employees
GROUP BY department_id HAVING COUNT(*) > 3;
```

13. Two techniques:

```
-- LEFT JOIN + IS NULL
SELECT c.* FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
WHERE o.order_id IS NULL;

-- NOT EXISTS
SELECT c.* FROM customers c
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id =
c.customer_id);
```

Expected: “NoOrders Inc.”

14.

```
SELECT order_id, SUM(quantity * unit_price) AS order_total
FROM order_items GROUP BY order_id;
```

15.

```
SELECT * FROM employees WHERE salary > (SELECT AVG(salary) FROM
employees);
```

16.

```
SELECT o.order_id, c.name, COUNT(oi.order_item_id) AS item_count
FROM orders o
JOIN customers c ON c.customer_id = o.customer_id
LEFT JOIN order_items oi ON oi.order_id = o.order_id
GROUP BY o.order_id, c.name;
```

17.

```
SELECT p.* FROM products p
WHERE NOT EXISTS (SELECT 1 FROM order_items oi WHERE oi.product_id =
p.product_id);
```

18.

```
SELECT department_id, MAX(salary) AS top_salary
FROM employees GROUP BY department_id;
```

19.

```
SELECT customer_id, COUNT(*) AS order_count
FROM orders GROUP BY customer_id HAVING COUNT(*) > 1;
```

20.

```
SELECT e.* FROM employees e
JOIN employees m ON e.manager_id = m.employee_id
```

```
WHERE m.name = 'Alice Chen';
```

21.

```
SELECT name, department_id, salary,
       DENSE_RANK() OVER (PARTITION BY department_id ORDER BY salary
DESC) AS rnk
FROM employees;
```

22.

```
SELECT * FROM (
  SELECT DISTINCT salary, DENSE_RANK() OVER (ORDER BY salary DESC)
AS rnk
  FROM employees
) x WHERE rnk = 2;
```

Using DENSE_RANK (not ROW_NUMBER) means a salary tied for 1st doesn't accidentally "use up" rank 2.

23.

```
SELECT * FROM (
  SELECT customer_id, order_date,
         ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY
order_date DESC) AS rn
  FROM orders
) x WHERE rn = 1;
```

24.

```
WITH RECURSIVE chain AS (
  SELECT employee_id, name, manager_id, 0 AS depth
  FROM employees WHERE name = 'Jack Nguyen'
  UNION ALL
  SELECT e.employee_id, e.name, e.manager_id, c.depth + 1
  FROM employees e JOIN chain c ON e.employee_id = c.manager_id
)
SELECT * FROM chain ORDER BY depth;
```

25.

```
SELECT payment_id, paid_at, amount,
       SUM(amount) OVER (ORDER BY paid_at ROWS BETWEEN UNBOUNDED
PRECEDING AND CURRENT ROW) AS running_total
FROM payments
WHERE paid_at IS NOT NULL
ORDER BY paid_at;
```

Note: filtering out paid_at IS NULL first — otherwise the ordering/window is undefined for that row.

26.

```
SELECT e1.* FROM employees e1
WHERE EXISTS (
  SELECT 1 FROM employees e2
  WHERE e2.salary = e1.salary AND e2.employee_id <> e1.employee_id
);
```

Expected: Bob/Carol pair and Grace/Henry pair.

27.

```
WITH monthly AS (
  SELECT DATE_FORMAT(order_date, '%Y-%m') AS month, COUNT(*) AS
order_count
  FROM orders GROUP BY DATE_FORMAT(order_date, '%Y-%m')
```

```

    )
    SELECT month, order_count,
           order_count - LAG(order_count) OVER (ORDER BY month) AS
mom_change
    FROM monthly;

```

28.

```

WITH delta_orders AS (
    SELECT order_date,
           ROW_NUMBER() OVER (ORDER BY order_date) AS rn
    FROM orders WHERE customer_id = 4
),
islands AS (
    SELECT order_date, DATE_SUB(order_date, INTERVAL rn DAY) AS
island_key
    FROM delta_orders
)
SELECT island_key, COUNT(*) AS streak_length, MIN(order_date) AS
streak_start, MAX(order_date) AS streak_end
FROM islands GROUP BY island_key ORDER BY streak_length DESC;

```

29.

```

SELECT * FROM (
    SELECT *, ROW_NUMBER() OVER (PARTITION BY employee_id ORDER BY
changed_at DESC) AS rn
    FROM employee_history
    WHERE old_salary <> new_salary
) x WHERE rn = 1;

```

30.

```

SELECT * FROM (
    SELECT customer_id, status, changed_at,
           ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY
changed_at DESC) AS rn
    FROM customer_status_history
) x WHERE rn = 1;

```

Note: this correctly returns customer 6's latest row even though it's a duplicate ACTIVE status — "latest recorded," not "latest distinct change." If you needed the latter, you'd need to also compare against LAG(status) and filter out rows where the status didn't actually change before ranking.

31.

```

SELECT department_id, AVG(salary) AS dept_avg
FROM employees
GROUP BY department_id
HAVING AVG(salary) > (SELECT AVG(salary) FROM employees);

```

(This is the exact pattern from your earlier lesson — non-correlated subquery in HAVING.)

32.

```

SELECT * FROM (
    SELECT *, ROW_NUMBER() OVER (PARTITION BY category_id ORDER BY
price DESC) AS rn
    FROM products
) x WHERE rn <= 2;

```

33.

```

SELECT o.customer_id
FROM orders o

```

```

JOIN order_items oi ON oi.order_id = o.order_id
JOIN products p ON p.product_id = oi.product_id
WHERE p.category_id = 2
GROUP BY o.customer_id
HAVING COUNT(DISTINCT p.product_id) = (SELECT COUNT(*) FROM products
WHERE category_id = 2);

```

34.

```

SELECT e1.* FROM employees e1
JOIN employees e2 ON e1.salary = e2.salary
AND e1.hire_date = e2.hire_date
AND e1.employee_id <> e2.employee_id;

```

Expected: Grace Park & Henry Adams.

35.

```

SELECT o.order_id,
       COALESCE(SUM(oi.quantity * oi.unit_price), 0) AS items_total,
       COALESCE(p.amount, 0) AS paid_amount
FROM orders o
LEFT JOIN order_items oi ON oi.order_id = o.order_id
LEFT JOIN payments p ON p.order_id = o.order_id
GROUP BY o.order_id, p.amount
HAVING items_total <> paid_amount;

```

36.

```

SELECT * FROM orders
WHERE order_date >= '2024-01-01' AND order_date < '2025-01-01';

```

37. (Written answer, no query): Without an index on department_id, EXPLAIN would show type = ALL (full table scan), key = NULL, and rows equal to the full row count of employees — MySQL must check every row. With an index on department_id, type would become ref, key would show the index name, and rows would drop to roughly the count of employees in department 3 — MySQL seeks directly to matching rows instead of scanning everything.

38.

```

SELECT e.* FROM employees e
WHERE NOT EXISTS (SELECT 1 FROM employee_history h WHERE
h.employee_id = e.employee_id);

```

39.

```

SELECT p.* FROM products p
WHERE NOT EXISTS (SELECT 1 FROM order_items oi WHERE oi.product_id =
p.product_id);

```

Written explanation: if order_items.product_id could be NULL, NOT IN (SELECT product_id FROM order_items) would break entirely — one NULL in the subquery's result set makes every <> comparison evaluate to UNKNOWN, so the whole NOT IN expression returns UNKNOWN for every row and the query returns zero rows regardless of the real data. NOT EXISTS has no such trap because it checks row existence per correlated match, not list membership.

40.

```

SELECT department_id,
       SUM(salary) AS dept_total,
       SUM(salary) / (SELECT SUM(salary) FROM employees) * 100 AS
pct_of_company
FROM employees
GROUP BY department_id;

```

PART 25 — DEBUGGING LAB (find the bug before reading the fix)

Bug 1

```
SELECT c.name, o.order_id FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
WHERE o.status = 'COMPLETED';
```

What's wrong: putting the right-table filter in WHERE silently turns the LEFT JOIN into an INNER JOIN — any customer with no matching order (o.status is NULL) fails o.status = 'COMPLETED' and gets dropped. *Fix:* move the condition into the ON clause: ON o.customer_id = c.customer_id AND o.status = 'COMPLETED'.

Bug 2

```
SELECT department_id, name, salary FROM employees GROUP BY
department_id;
```

What's wrong: name and salary aren't aggregated and aren't in the GROUP BY — MySQL (in default ONLY_FULL_GROUP_BY mode) will error, or if that mode is off, silently return an arbitrary row per group. *Fix:* either aggregate (MAX(salary)) or add the columns to GROUP BY, or use a window function instead if you need the row-level detail.

Bug 3

```
SELECT department_id, AVG(salary) FROM employees WHERE AVG(salary) >
70000 GROUP BY department_id;
```

What's wrong: aggregate functions can't be used in WHERE — WHERE runs before grouping/aggregation happens (see the logical order in Part 2). *Fix:* use HAVING AVG(salary) > 70000 instead.

Bug 4

```
SELECT * FROM employees WHERE department_id NOT IN (SELECT
department_id FROM employee_history);
```

What's wrong: if employee_history.department_id (or any joined-in NULL) can be NULL, this returns zero rows unexpectedly (the classic NOT IN + NULL trap). *Fix:* add WHERE department_id IS NOT NULL inside the subquery, or rewrite using NOT EXISTS.

Bug 5

```
WITH RECURSIVE chain AS (
  SELECT employee_id, manager_id FROM employees WHERE employee_id
= 10
  UNION ALL
  SELECT e.employee_id, e.manager_id FROM employees e, chain
)
SELECT * FROM chain;
```

What's wrong: no join condition linking e to chain — this is an accidental cross join in the recursive step, and it has no termination logic tied to the actual hierarchy, risking runaway recursion. *Fix:* FROM employees e JOIN chain c ON e.employee_id = c.manager_id.

Bug 6

```
SELECT *, ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY
order_date DESC) AS rn
FROM orders WHERE rn = 1;
```

What's wrong: can't filter on a window function alias in the same-level WHERE — window functions are computed after WHERE runs. *Fix:* wrap it in a subquery/CTE and filter `rn = 1` in the outer query.

Bug 7

```
SELECT customer_id, SUM(amount) OVER (PARTITION BY customer_id) FROM
orders GROUP BY customer_id;
```

What's wrong: mixing a window function with GROUP BY on the same non-aggregated query without proper structure is a common confusion — here it's actually more likely to error or behave unexpectedly, because you're trying to aggregate the group *and* window-partition on top of it inconsistently. *Fix:* pick one: either plain GROUP BY SUM(amount) (no window function needed for a single per-group total), or drop GROUP BY entirely and rely on the window function alone across ungrouped detail rows.

Bug 8

```
SELECT * FROM employees e WHERE e.salary = (SELECT salary FROM
employees WHERE department_id = e.department_id);
```

What's wrong: the subquery can return more than one row per department (multiple employees), and a scalar subquery used with = requires exactly one row — this will error at runtime ("Subquery returns more than 1 row") for any department with more than one employee. *Fix:* use IN instead of =, or aggregate (e.g., MAX(salary)) if you meant "top earner."

Bug 9

```
SELECT order_date, LAST_VALUE(amount) OVER (ORDER BY order_date)
FROM payments;
```

What's wrong: with the default window frame (RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW), LAST_VALUE returns the *current row's* value, not the actual last row of the partition — a very common misunderstanding. *Fix:* explicitly widen the frame: LAST_VALUE(amount) OVER (ORDER BY order_date ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING).

Bug 10

```
SELECT * FROM orders WHERE YEAR(order_date) = 2024 AND customer_id =
4;
```

What's wrong: not a syntax error, but YEAR(order_date) is not sargable — even with an index on order_date, MySQL must evaluate the function on every row, ignoring the index. *Fix:* WHERE order_date >= '2024-01-01' AND order_date < '2025-01-01' AND customer_id = 4.

PART 26 — QUERY OPTIMIZATION LAB

Case 1 — Function on indexed column

```
-- Slow
SELECT * FROM orders WHERE YEAR(order_date) = 2024;
-- EXPLAIN shows: type = ALL, key = NULL, rows ≈ full table
-- Fix
SELECT * FROM orders WHERE order_date >= '2024-01-01' AND order_date
< '2025-01-01';
-- With an index on order_date: type = range, key = idx_order_date,
rows shrinks to the 2024 subset
```

Index recommendation: CREATE INDEX idx_order_date ON orders(order_date);

Case 2 — Leading wildcard

```
-- Slow
SELECT * FROM customers WHERE name LIKE '%Corp%';
-- No index can help a leading/trailing wildcard scan
-- Fix: use FULLTEXT search if this is a real text-search need
ALTER TABLE customers ADD FULLTEXT(name);
SELECT * FROM customers WHERE MATCH(name) AGAINST('Corp');
```

Case 3 — Missing index on join column

```
-- Slow at scale
SELECT * FROM orders o JOIN customers c ON c.customer_id =
o.customer_id WHERE c.status = 'ACTIVE';
-- EXPLAIN shows a full scan on orders if orders.customer_id has no
index
-- Fix
CREATE INDEX idx_orders_customer ON orders(customer_id);
```

Case 4 — Wrong composite index column order

```
-- Index exists as (status, customer_id) but query filters mainly by
customer_id
CREATE INDEX idx_bad ON orders(status, customer_id);
SELECT * FROM orders WHERE customer_id = 4; -- can't use idx_bad
efficiently – customer_id isn't leftmost
-- Fix: reorder to match the leading filter column
CREATE INDEX idx_good ON orders(customer_id, status);
```

Case 5 — SELECT * defeating a covering index

```
-- Index covers (customer_id, order_date) but SELECT * forces a row
lookup anyway
CREATE INDEX idx_cust_date ON orders(customer_id, order_date);
SELECT * FROM orders WHERE customer_id = 4; -- Extra:
NULL (row lookup needed)
SELECT customer_id, order_date FROM orders WHERE customer_id = 4; -
- Extra: Using index (covering!)
```

Case 6 — Correlated subquery vs JOIN at scale

```
-- Slower on large tables – recomputed per row
SELECT * FROM employees e
WHERE salary > (SELECT AVG(salary) FROM employees e2 WHERE
e2.department_id = e.department_id);
-- Faster – aggregate computed once, then joined
SELECT e.* FROM employees e
JOIN (SELECT department_id, AVG(salary) avg_sal FROM employees GROUP
BY department_id) d
ON d.department_id = e.department_id
WHERE e.salary > d.avg_sal;
```

Case 7 — OR preventing index use

```
-- May force a full scan even with indexes on both columns
individually
SELECT * FROM orders WHERE customer_id = 4 OR status = 'PENDING';
-- Fix: split into a UNION of two indexed queries
SELECT * FROM orders WHERE customer_id = 4
UNION
SELECT * FROM orders WHERE status = 'PENDING';
```

Case 8 — Offset pagination degrading at depth

```
-- Gets slower as OFFSET grows – MySQL scans and discards the first
```

50,000 rows every time

```
SELECT * FROM orders ORDER BY order_id LIMIT 20 OFFSET 50000;
```

-- Fix: keyset pagination – remember the last order_id from the previous page

```
SELECT * FROM orders WHERE order_id > 50000 ORDER BY order_id LIMIT 20;
```

(Continue to Part 3 for the interview question bank, cheat sheets, interview traps, capstone project, and the 30/60/90 day plan.)